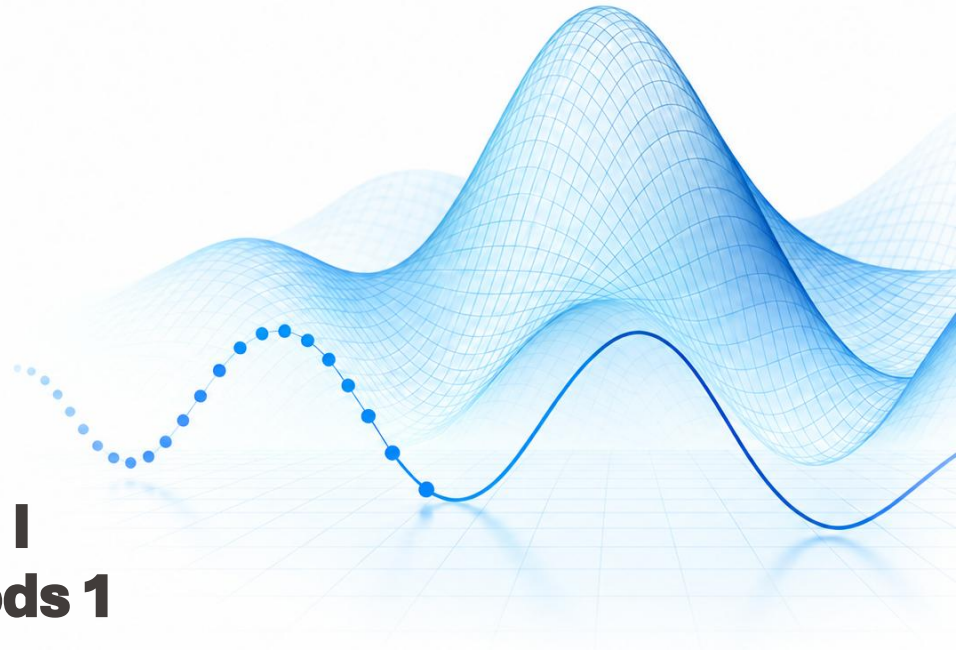


IE2106 – Engineering Math I
Week 7: Numerical Methods 1

Mohammad NIKOO
Nanyang Assistant Professor
Singapore NRF Fellow
EEE, NTU

IE2106: Overview

NTU SINGAPORE

Part I (Week 1–6)

- Fundamentals of Probability
- Conditional Probability
- Random variables (pmf & pdf)
- Expected value, Variance, Standard Deviation
- Important distributions (Normal, Binomial, Poisson)
- Sample mean ($\overline{X}$) and variance S^2
- Chi squared and t distributions
- Estimation of μ and σ
- Hypothesis

Part II (Week 7–13)

- **Numerical methods**
  1) Algebraic equations
  2) Differential equations
- **Laplace transform**
  1) Concept and properties
  2) Solving differential equations with Laplace
- **Partial differential equations**
  1) Direct integration 2) Separation of variables
  3) Laplace transform

IE2106: Part II in a Glance

Numerical Methods
Week 7–8

In math, we usually talk about EXACT solutions, for example:

$$x^2 - 3x + 2 = 0 \Rightarrow x = 1, 2$$

$$\sin(1) = 0.8414709848078965 \ldots$$

But it is often NOT PRACTICAL to work with exact solutions!

Because..

1) Computers have limited storage and must truncate numbers.
2) It is mathematically IMPOSSIBLE to exactly solve many equations.

Laplace Transformation
Week 9–11

Laplace Transform

$$f(t) \xrightarrow{\mathcal{L}} F(s)$$

Time domain — Laplace domain

Laplace Transform

$$F(s) \xrightarrow{\mathcal{L}^{-1}} f(t)$$

Laplace domain — Time domain

Difficult problems in time domain become quite easy in Laplace domain!

Partial Differential Equations (PDE)
Week 12–13

We learn how to solve differential equations with more than one variable

For example, the wave equation:

$$u_{tt} = c^2 u_{xx}$$

Where u_{tt} is the second derivative of $u(x,t)$ w.r.t. time (t), and u_{xx} is the second derivative of $u(x,t)$ w.r.t. x

We will learn three methods:

1) Direct integration
2) Separation of variables
3) Laplace transform

Today's Objectives

Approximation of Functions by Polynomials

Digital computers are very good in Arithmetic calculations

$$a + b, \qquad a - b, \qquad a \times b$$

That's why it is EASY for a computer to calculate polynomials

$$p_3(x) = ax^3 + bx^2 + cx + d$$

However, it is DIFFICULT for them to compute other functions, such as

$$\sin(x), \qquad e^x, \qquad \ln(x)$$

We want to approximate "complicated" functions by polynomials:

$$\sin(x) \cong ax^2 + bx + c$$

We will learn Newton's Divided Difference Interpolation

Numerical Solution of Equations

Very limited types of equations have analytical solutions

$$ax^2 + bx + c = 0 \quad \Rightarrow \quad x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Most equations don't! For example:

$$x - \cos(x) = 0$$

We want to approximately solve such "complicated" equations using recursive sequences. We will learn how to write a recursive sequence that converges to the root of the equation

$$x_{n+1} = \cos(x_n)$$

We call this technique the fixed-point iteration.

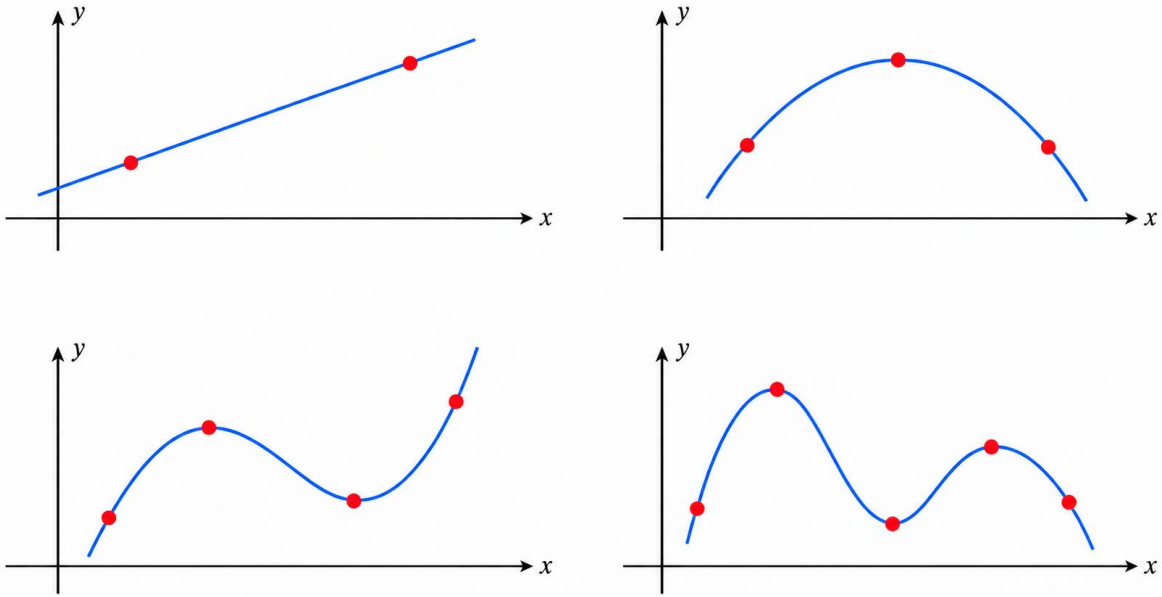
PART I

HOW TO APPROXIMATE “COMPLICATED” FUNCTIONS BY POLYNOMIALS?

Polynomial Interpolation

CORE IDEA

Given $n + 1$ points with distinct x-coordinates, there exists a polynomial of degree at most n that passes through all of them.



Polynomial Interpolation

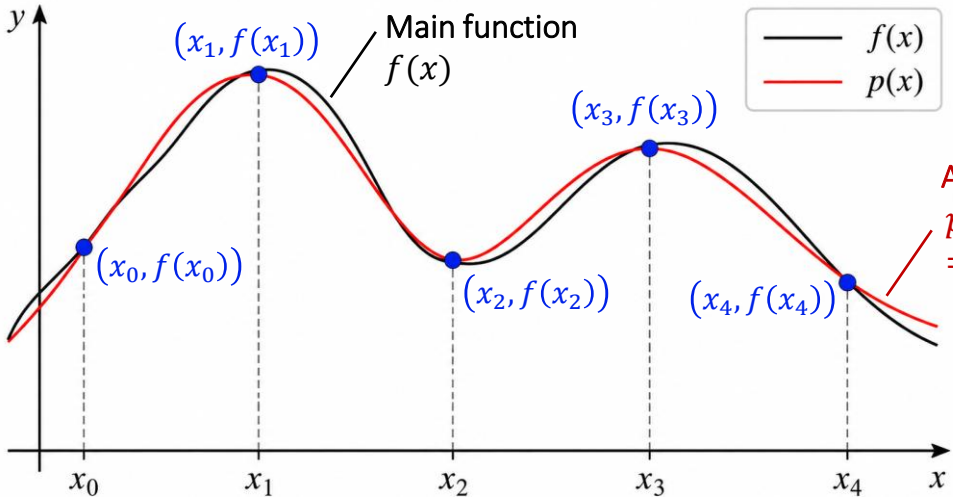
To approximate a function $f(x)$ using a polynomial of degree at most n, we select $n+1$ points from $f(x)$:

$$(x_0, f(x_0)), \quad (x_1, f(x_1)), \quad \dots \quad , (x_n, f(x_n))$$

and construct a polynomial that passes through all of them.

$$p(x) = a_n x^n + \cdots + a_1 x + a_0$$

$p(x)$ serves as an approximation for $f(x)$: $f(x) \approx p(x)$



Approximated polynomial (degree 4)

$p(x)$
$= a_4 x^4 + a_3 x^3 + a_2 x^2 + a_1 x + a_0$

Polynomial Interpolation: Concept

To approximate a function $f(x)$ using a polynomial of degree at most n, we select $n+1$ points from $f(x)$:

$$(x_0, f(x_0)), \quad (x_1, f(x_1)), \quad \dots \quad , (x_n, f(x_n))$$

and construct a polynomial that passes through all of them.

$$p(x) = a_n x^n + \cdots + a_1 x + a_0$$

$p(x)$ serves as an approximation for $f(x)$: $f(x) \approx p(x)$

The only question is how to find the coefficients $a_0, a_1, \dots, a_n$

There is a straightforward way to find the coefficients:

How many unknowns do we have? As many as the coefficients: $n+1$

How many equations can we write? As many as the interpolation points: $n+1$

So, we can form a system of $n+1$ equations and solve for all $n+1$ coefficients.

Example: Approximate $y = \sin(\pi x)$ using the simplest polynomial that interpolates the points at $x = 0, \frac{1}{4}, \frac{1}{2}$

We use the following points:

$$(0,0), \qquad \left(\frac{1}{4}, \sin\left(\frac{\pi}{4}\right)\right) = \left(\frac{1}{4}, \frac{\sqrt{2}}{2}\right), \qquad \left(\frac{1}{2}, \sin\left(\frac{\pi}{2}\right)\right) = \left(\frac{1}{2}, 1\right)$$

We can find a second degree polynomial that interpolates through these points: $p(x) = a_2x^2 + a_1x + a_0$

We can form a system of equations to find the coefficients a_0, a_1, a_2

$$\begin{cases} p(0) = 0 \\ p\left(\frac{1}{4}\right) = \frac{\sqrt{2}}{2} \\ p\left(\frac{1}{2}\right) = 1 \end{cases} \Rightarrow \begin{cases} a_2 \cdot 0^2 + a_1 \cdot 0 + a_0 = 0 \\ a_2 \cdot \left(\frac{1}{4}\right)^2 + a_1 \cdot \left(\frac{1}{4}\right) + a_0 = \frac{\sqrt{2}}{2} \\ a_2 \cdot \left(\frac{1}{2}\right)^2 + a_1 \cdot \left(\frac{1}{2}\right) + a_0 = 1 \end{cases} \Rightarrow \begin{cases} a_0 = 0 \\ \frac{1}{16}a_2 + \frac{1}{4}a_1 + a_0 = \frac{\sqrt{2}}{2} \\ \frac{1}{4}a_2 + \frac{1}{2}a_1 + a_0 = 1 \end{cases} \Rightarrow \begin{cases} a_0 = 0 \\ a_1 = 4\sqrt{2} - 2 \\ a_2 = 8 - 8\sqrt{2} \end{cases}$$

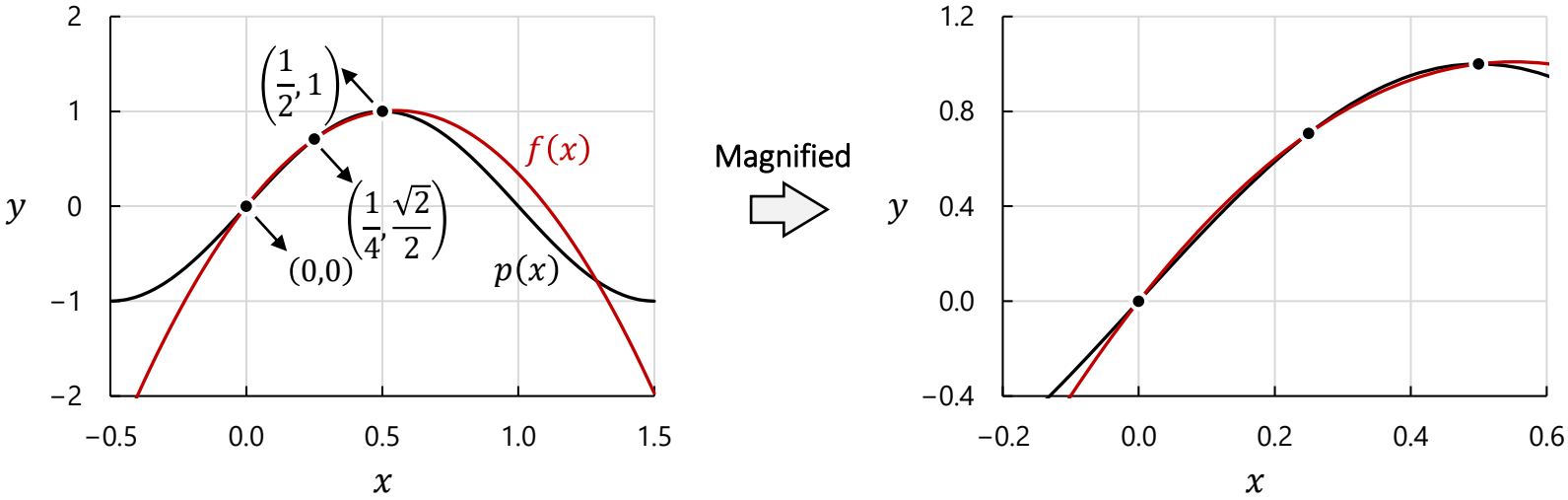
We can therefore write the polynomial as follows:

$$p(x) = a_2x^2 + a_1x + a_0 \quad \Rightarrow \quad \boxed{p(x) = \left(8 - 8\sqrt{2}\right)x^2 + \left(4\sqrt{2} - 2\right)x}$$

Polynomial Interpolation

Example: Approximate $y = \sin(\pi x)$ using the simplest polynomial that interpolates the points at $x = 0, \frac{1}{4}, \frac{1}{2}$

$p(x) = (8 - 8\sqrt{2})x^2 + (4\sqrt{2} - 2)x$ serves as an approximation for $f(x) = \sin(\pi x)$



What do you learn from this?

The problem with systems of equations method

The goal is to approximate a function $f(x)$ using a polynomial.

To do so, we select $n+1$ points from $f(x)$:

$$(x_0, f(x_0)), \quad (x_1, f(x_1)), \quad \dots \quad , (x_n, f(x_n))$$

and find a polynomial that interpolates through those points: $p(x) = a_n x^n + \cdots + a_1 x + a_0$

The main objective is finding the coefficients $a_0, a_1, \dots, a_n$

We just saw that we could find the coefficients by forming a system of equations.

But this method is not ideal. Why?

Because..

1) It is computationally expensive (especially if we have many points!)

2) If we want to add an extra interpolation point, we generally need to solve the entire system again.

So we will introduce a smart way of finding this interpolating polynomial!

Newton's Divided Difference Interpolation

We start from a three-point interpolation to explain the technique.

We want to approximate a function $f(x)$ using a second-degree polynomial $p_2(x)$.

We select 3 points from $f(x)$:

$$(x_0, f_0), \qquad (x_1, f_1), \qquad (x_2, f_2)$$

Step 1: Compute the first-divided differences

$$f[x_0, x_1] = \frac{f_1 - f_0}{x_1 - x_0}, \qquad f[x_1, x_2] = \frac{f_2 - f_1}{x_2 - x_1}$$

Step 2: Compute the second-divided differences

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$$

Step 3: Writing the interpolating polynomial

$$p_2(x) = f_0 + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1)$$

Newton's Divided Difference Interpolation

We can extend the method to 4 points or more!

To approximate a function $f(x)$ using a third-degree polynomial $p_3(x)$, we select 4 points from $f(x)$:

$$(x_0, f_0), \qquad (x_1, f_1), \qquad (x_2, f_2), \qquad (x_3, f_3)$$

Step 1: Compute the first-divided differences

$$f[x_0, x_1] = \frac{f_1 - f_0}{x_1 - x_0}, \qquad f[x_1, x_2] = \frac{f_2 - f_1}{x_2 - x_1}, \qquad f[x_2, x_3] = \frac{f_3 - f_2}{x_3 - x_2}$$

Step 2: Compute the second-divided differences

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}, \qquad f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}$$

Step 3: Compute the third-divided differences

$$f[x_0, x_1, x_2, x_3] = \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0}$$

Step 4: Writing the interpolating polynomial

$$p_3(x) = f_0 + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2)$$

Newton's Divided Difference Interpolation

Newton's Divided Difference Interpolation method has two important advantages:

1) It is computationally less expensive compared to solving a system of equations.

2) An n-point interpolation can be extended to $n+1$-point interpolation.

New terms to be computed

3-points interpolation

First-divided differences

$$f[x_0,x_1]=\frac{f_1-f_0}{x_1-x_0},\qquad f[x_1,x_2]=\frac{f_2-f_1}{x_2-x_1}$$

Second-divided differences

$$f[x_0,x_1,x_2]=\frac{f[x_1,x_2]-f[x_0,x_1]}{x_2-x_0}$$

Writing the interpolating polynomial

$$p_2(x)=f_0+f[x_0,x_1](x-x_0)+f[x_0,x_1,x_2](x-x_0)(x-x_1)$$

4-points interpolation

First-divided differences

$$f[x_0,x_1]=\frac{f_1-f_0}{x_1-x_0},\qquad f[x_1,x_2]=\frac{f_2-f_1}{x_2-x_1},\qquad f[x_2,x_3]=\frac{f_3-f_2}{x_3-x_2}$$

Second-divided differences

$$f[x_0,x_1,x_2]=\frac{f[x_1,x_2]-f[x_0,x_1]}{x_2-x_0},\qquad f[x_1,x_2,x_3]=\frac{f[x_2,x_3]-f[x_1,x_2]}{x_3-x_1}$$

Third-divided differences

$$f[x_0,x_1,x_2,x_3]=\frac{f[x_1,x_2,x_3]-f[x_0,x_1,x_2]}{x_3-x_0}$$

Writing the interpolating polynomial

$$p_3(x)=f_0+f[x_0,x_1](x-x_0)+f[x_0,x_1,x_2](x-x_0)(x-x_1)+f[x_0,x_1,x_2,x_3](x-x_0)(x-x_1)(x-x_2)$$

Newton's Divided Difference Interpolation

Newton's Divided Difference Interpolation with $n+1$ points

x_i	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$f[x_0]$			
x_1	$f[x_1]$	$f[x_0,x_1]$		
x_2	$f[x_2]$	$f[x_1,x_2]$	$f[x_0,x_1,x_2]$	
x_3	$f[x_3]$	$f[x_2,x_3]$	$f[x_1,x_2,x_3]$	$f[x_0,x_1,x_2,x_3]$

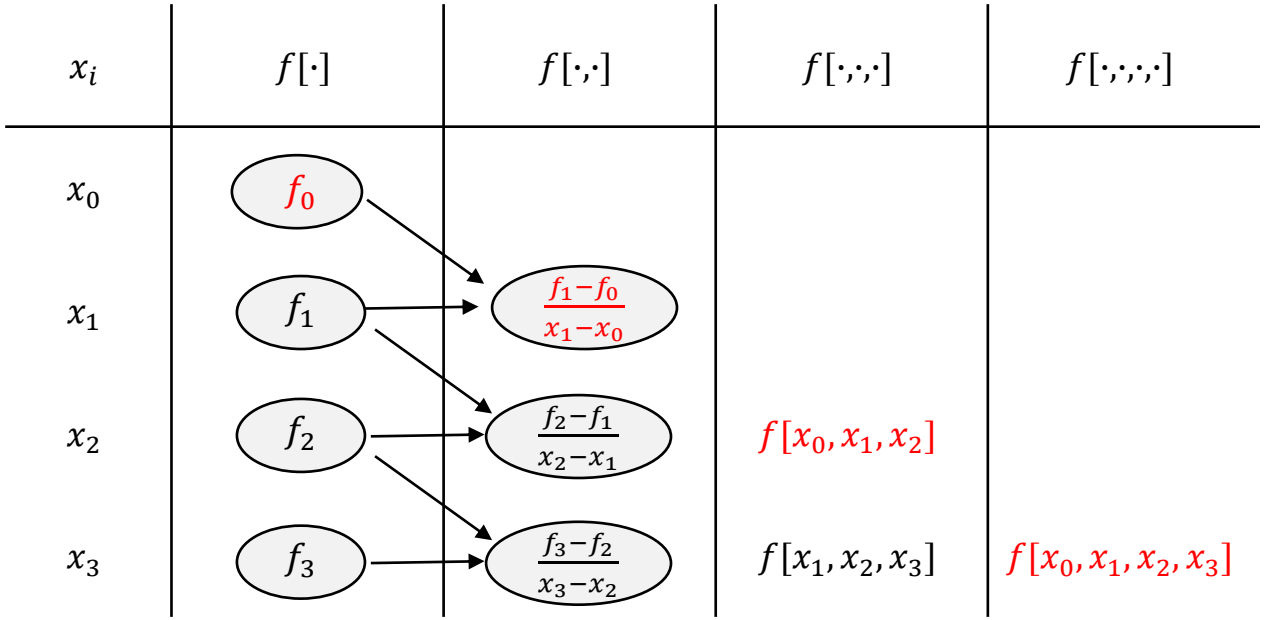
$$p(x) = f[x_0] + f[x_0,x_1](x-x_0) + f[x_0,x_1,x_2](x-x_0)(x-x_1) + f[x_0,x_1,x_2,x_3](x-x_0)(x-x_1)(x-x_2)$$

Newton's Divided Difference Interpolation

Newton's Divided Difference Interpolation with $n+1$ points

Hint:

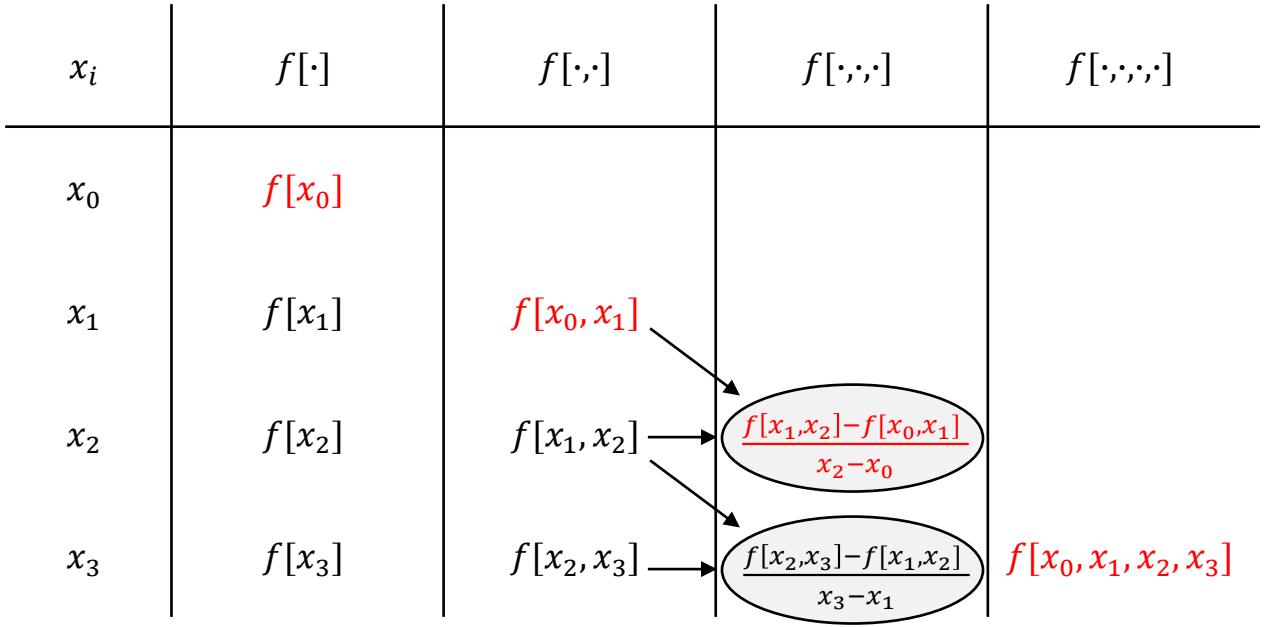
$f_i = f(x_i) = f[x_i]$



$$p(x) = f[x_0] + f[x_0,x_1](x-x_0) + f[x_0,x_1,x_2](x-x_0)(x-x_1) + f[x_0,x_1,x_2,x_3](x-x_0)(x-x_1)(x-x_2)$$

Newton's Divided Difference Interpolation

Newton's Divided Difference Interpolation with $n+1$ points



$$p(x) = f[x_0] + f[x_0,x_1](x-x_0) + f[x_0,x_1,x_2](x-x_0)(x-x_1) + f[x_0,x_1,x_2,x_3](x-x_0)(x-x_1)(x-x_2)$$

Newton's Divided Difference Interpolation

Newton's Divided Difference Interpolation with $n+1$ points

x_i	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$f[x_0]$			
x_1	$f[x_1]$	$f[x_0,x_1]$		
x_2	$f[x_2]$	$f[x_1,x_2]$	$f[x_0,x_1,x_2]$	
x_3	$f[x_3]$	$f[x_2,x_3]$	$f[x_1,x_2,x_3]$	$\frac{f[x_1,x_2,x_3]-f[x_0,x_1,x_2]}{x_3-x_0}$

$$p(x) = f[x_0] + f[x_0,x_1](x-x_0) + f[x_0,x_1,x_2](x-x_0)(x-x_1) + f[x_0,x_1,x_2,x_3](x-x_0)(x-x_1)(x-x_2)$$

Error of Polynomial Interpolation

Consider $f(x)$ to be $n+1$ times differentiable.

And let $p_n(x)$ be the n-th degree polynomial interpolating from the following $n+1$ points from f

$$(x_0, f_0), \quad (x_1, f_1), \quad (x_2, f_2), \quad \dots, \quad (x_n, f_n)$$

We can use the following formula to calculate an upper bound for the error of approximation

$$|f(x) - p_n(x)| \leq \frac{1}{(n+1)!} \left| \prod_{k=0}^{n} (x - x_k) \right| \max_{\xi} \left| f^{(n+1)}(\xi) \right|$$

Where $\min\{x, x_0, x_1, \dots, x_n\} \leq \xi \leq \max\{x, x_0, x_1, \dots, x_n\}$

What do we learn from this formula?

1) Increasing n can potentially improve the approximation because of the $\frac{1}{(n+1)!}$ term

2) Having x closer to x_k leads to smaller error because of $x - x_k$ term. ($x = x_k$ leads to zero error!)

3) Having x outside of the interpolation range ($x_0 \dots x_n$) would usually lead to a large error. Why?

Class Exercise

Final Exam – 2025-2026 Semester 2

Using Newton's divided difference interpolation method, derive the second-degree polynomial that interpolates $f(x) = \frac{1}{x}$ at points $x_0 = 1, x_1 = 2,$ and $x_2 = 3.$

Class Exercise

Final Exam – 2025-2026 Semester 2

Using Newton's divided difference interpolation method, derive the second-degree polynomial that interpolates $f(x) = \frac{1}{x}$ at points $x_0 = 1, x_1 = 2$, and $x_2 = 3$.

Here are the points: $(1,1)$, $\left(2,\frac{1}{2}\right)$, $\left(3,\frac{1}{3}\right)$

Step 1: Compute the first-divided differences

$$f[x_0,x_1] = \frac{f_1 - f_0}{x_1 - x_0} = \frac{\frac{1}{2} - 1}{2 - 1} = -\frac{1}{2}, \qquad f[x_1,x_2] = \frac{\frac{1}{3} - \frac{1}{2}}{3 - 2} = -\frac{1}{6}$$

Step 2: Compute the second-divided differences

$$f[x_0,x_1,x_2] = \frac{f[x_1,x_2] - f[x_0,x_1]}{x_2 - x_0} = \frac{\left(-\frac{1}{6}\right) - \left(-\frac{1}{2}\right)}{3 - 1} = \frac{1}{6}$$

Step 3: Writing the interpolating polynomial

$$p_2(x) = f_0 + f[x_0,x_1](x - x_0) + f[x_0,x_1,x_2](x - x_0)(x - x_1) = 1 - \frac{1}{2}(x - 1) + \frac{1}{6}(x - 1)(x - 2)$$

$$\Rightarrow \quad \boxed{p_2(x) = \frac{1}{6}x^2 - x + \frac{11}{6}}$$

Class Exercise

Final Exam – 2025-2026 Semester 2

Using Newton's divided difference interpolation method, derive the second-degree polynomial that interpolates $f(x) = \frac{1}{x}$ at points $x_0 = 1, x_1 = 2,$ and $x_2 = 3.$

Follow-up question 1: Calculate the upper bound of error at $x = 2.5$

We can use the following formula

$$|f(x) - p_n(x)| \leq \frac{1}{(n+1)!} \left| \prod_{k=0}^{n} (x - x_k) \right| \max_{\xi} \left| f^{(n+1)}(\xi) \right|$$

We have all the parameters:

$n = 2$

$x = 2.5, x_0 = 1, x_1 = 2, x_2 = 3$

$\min\{2.5,1,2,3\} \leq \xi \leq \max\{2.5,1,2,3\}$ resulting in $1 \leq \xi \leq 3$

$f^{(3)} = -\frac{6}{x^4} \Rightarrow \max_{\xi} \left| f^{(n+1)}(\xi) \right| = \max_{1 \leq \xi \leq 3} \left| -\frac{6}{x^4} \right| = \max_{1 \leq \xi \leq 3} \left| \frac{6}{x^4} \right| = 6$

So we write:

$$|f(x) - p_2(x)| \leq \frac{1}{3!} |(2.5 - 1)(2.5 - 2)(2.5 - 3)| \cdot 6 = 0.375$$

Class Exercise

Final Exam – 2025-2026 Semester 2

Using Newton's divided difference interpolation method, derive the second-degree polynomial that interpolates $f(x) = \frac{1}{x}$ at points $x_0 = 1, x_1 = 2,$ and $x_2 = 3.$

Follow-up question 1: Calculate the upper bound of error at $x = 2.5$

Follow-up question 2: Calculate the exact absolute error at $x = 2.5$

We can calculate the exact value of the function f at $x = 2.5$

$$f(2.5) = \frac{1}{2.5} = 0.4$$

We can also calculate the value of the interpolation polynomial $p_2(x) = \frac{1}{6}x^2 - x + \frac{11}{6}$

$$p_2(2.5) = \frac{1}{6}2.5^2 - 2.5 + \frac{11}{6} = 0.375$$

We can now calculate the exact absolute error

$$|f(2.5) - p_2(2.5)| = |0.4 - 0.375| = 0.025$$

This is lower than the upper bound that we calculated: $|f(x) - p_2(x)| \leq \frac{1}{3!}|(2.5-1)(2.5-2)(2.5-3)| \cdot 6 = 0.375$

Definition of Error

Suppose $\tilde{a}$ approximates a

ABSOLUTE ERROR

The absolute error in $\tilde{a}$ is

$$\epsilon = |a - \tilde{a}|$$

RELATIVE ERROR

The relative error in $\tilde{a}$ is

$$r = \frac{\epsilon}{|a|} = \left|\frac{a - \tilde{a}}{a}\right|$$

Important statement that we will use down the way:

A number which is correct to n significant digits has a relative error $|r| < 0.5 \times 10^{1-n}$

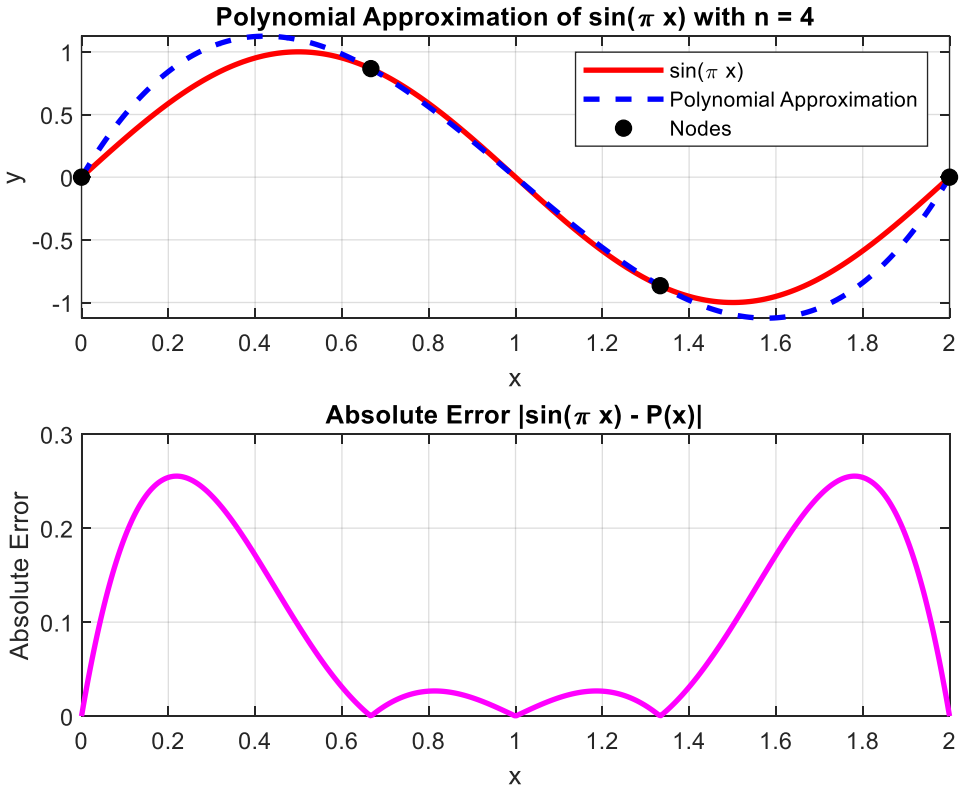
For example, $\tilde{a} = 3.14$ approximate $a = \pi$ to three significant digits, and we have

$$|r| = \left|\frac{a - \tilde{a}}{a}\right| = \left|\frac{3.1415\cdots - 3.14}{3.1415\cdots}\right| \cong 0.00050696 < 0.005$$

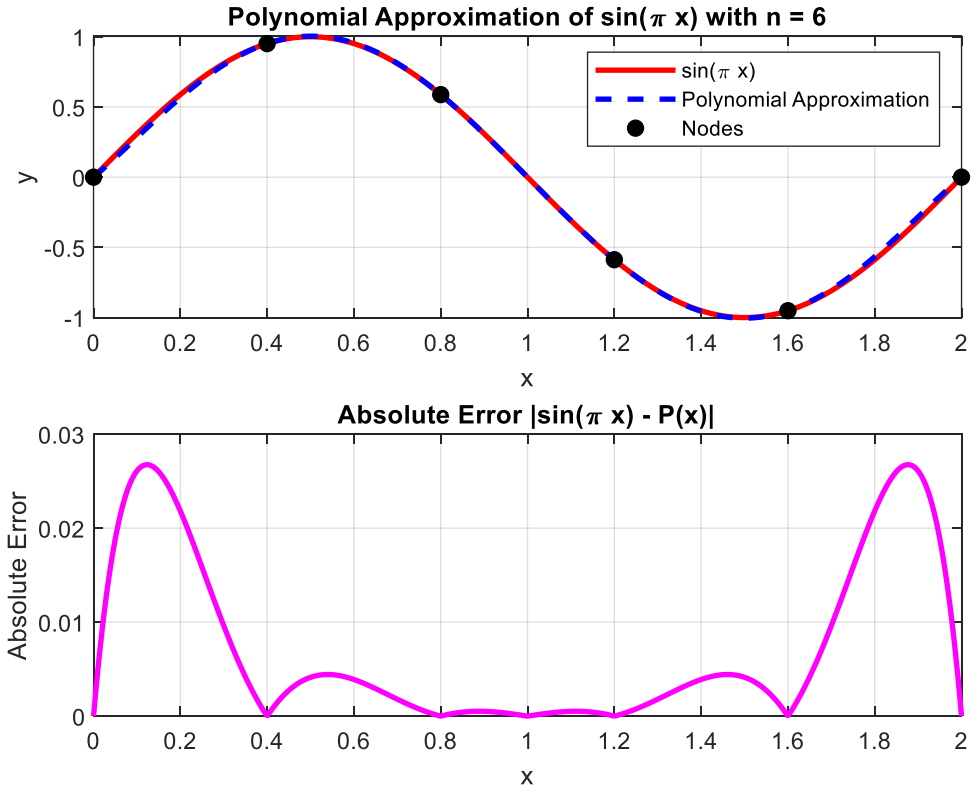
$$\downarrow$$

$$|r| < 0.5 \times 10^{1-3}$$

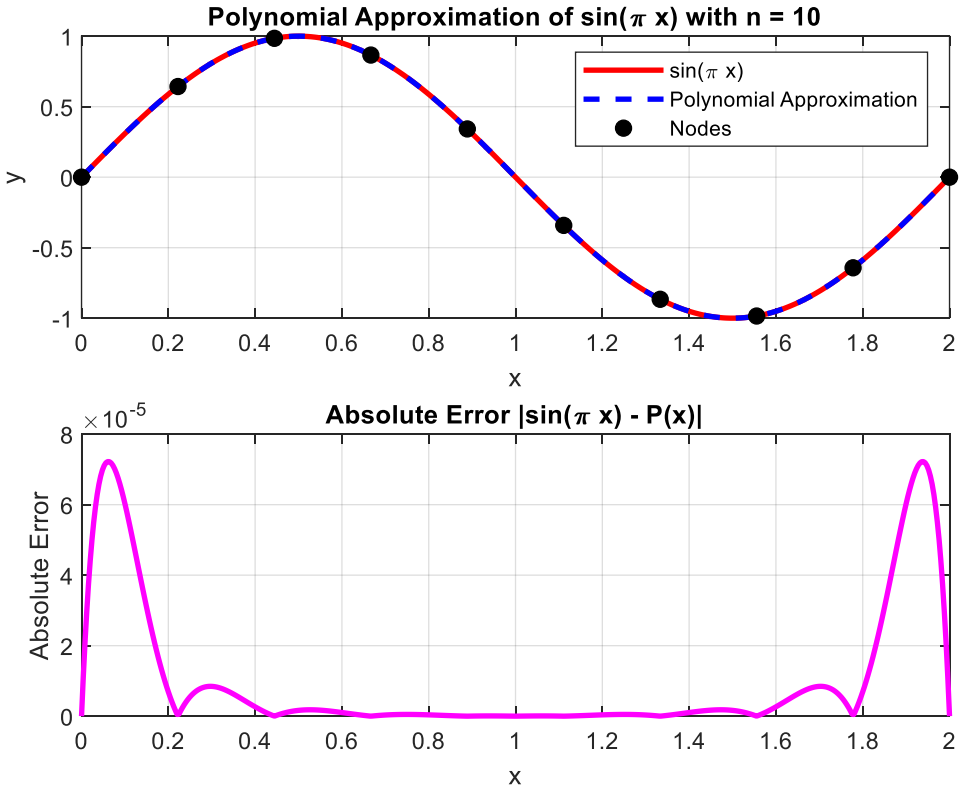
Practical Examples in Matlab (Excluded from Exam)



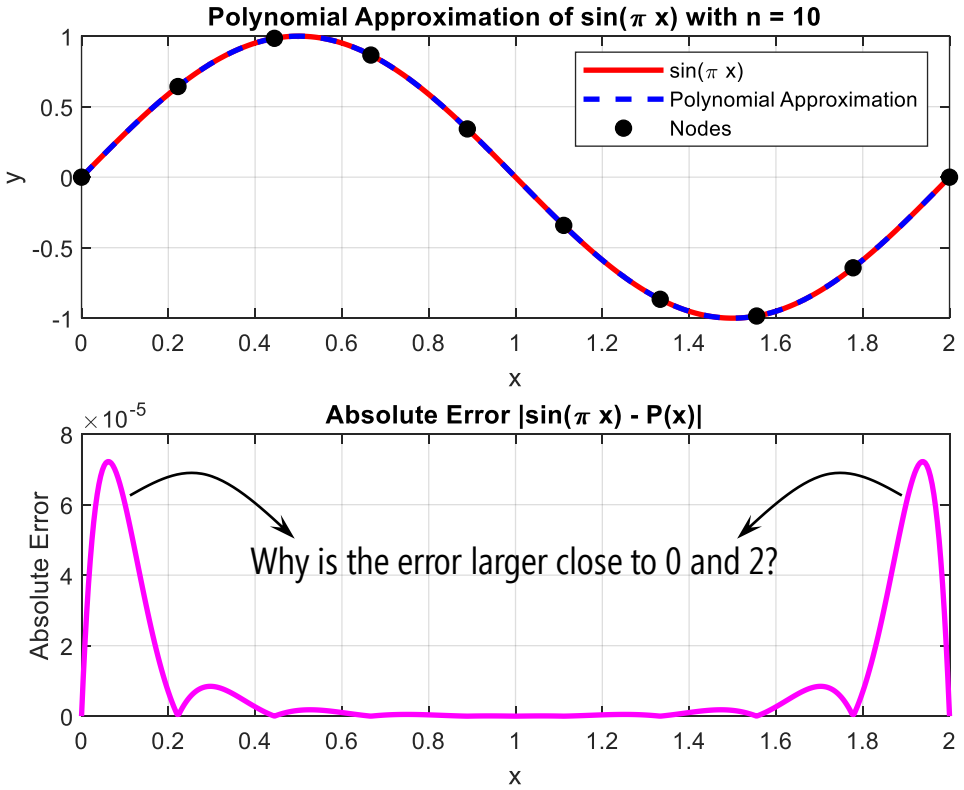
Practical Examples in Matlab (Excluded from Exam)



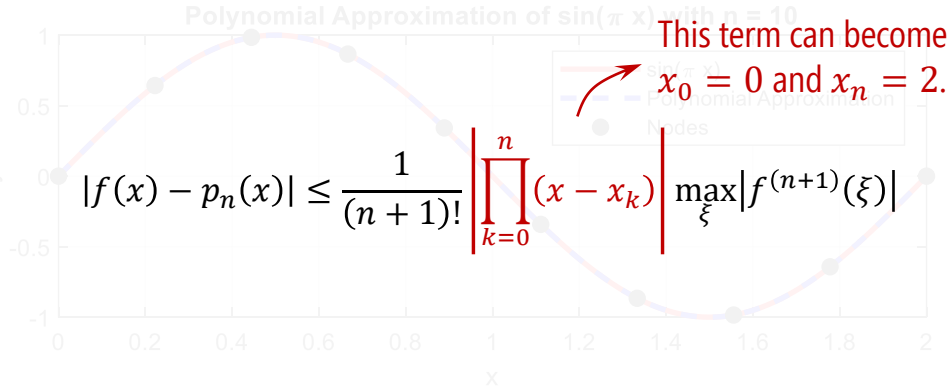
Practical Examples in Matlab (Excluded from Exam)



Practical Examples in Matlab (Excluded from Exam)

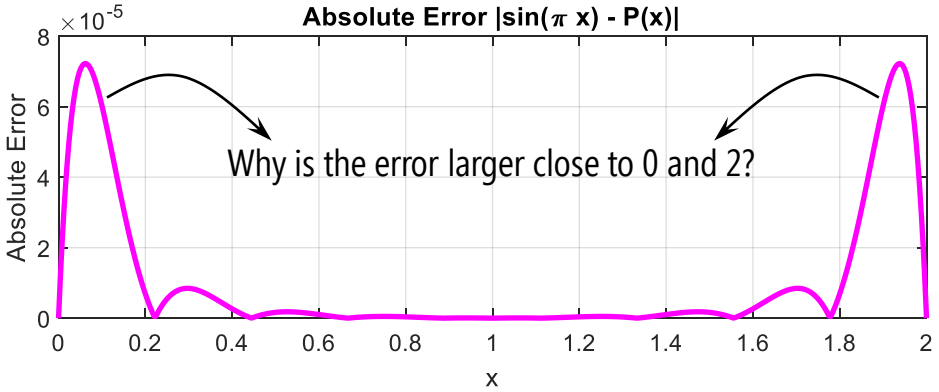


Practical Examples in Matlab (Excluded from Exam)



This term can become relatively large near $x_0 = 0$ and $x_n = 2$.

$$|f(x) - p_n(x)| \leq \frac{1}{(n+1)!} \left| \prod_{k=0}^{n} (x - x_k) \right| \max_{\xi} \left| f^{(n+1)}(\xi) \right|$$



PART II

HOW TO APPROXIMATELY SOLVE "COMPLICATED" EQUATIONS?

Sequences: Review

There are two ways of describing a sequence

1) Explicit

For example, $a_n = 1 + \frac{1}{n} + (-1)^{n+1}$

In this case we can compute a_n for any n directly using the formula, for example: $a_{100} = 1 + \frac{1}{100} + (-1)^{101} = 0.01$

2) Recursive

For example, the Fibonacci sequence: $F_n = F_{n-1} + F_{n-2}, F_0 = 0, F_1 = 1$

Using the recursive definition, F_n is calculated from previously computed terms:

$$F_2 = F_0 + F_1 = 1$$
$$F_3 = F_1 + F_2 = 1 + 1 = 2$$
$$F_4 = F_2 + F_3 = 1 + 2 = 3$$
$$\dots$$

Our goal is to use recursive sequences to solve equations!

Fixed-Point Iteration Method: The Concept

Find the convergence point of the following sequence

$$x_{n+1} = \sqrt[5]{1 - x_n}\,, \quad x_0 = 0.5$$

Method 1: Approximation

We can compute the terms of the sequence using the given equation

$$x_0 = 0.5$$
$$x_1 = \sqrt[5]{1 - 0.5} = 0.870551$$
$$x_2 = \sqrt[5]{1 - 0.870551} = 0.664385$$
$$x_3 = \sqrt[5]{1 - 0.664385} = 0.803837$$
$$x_4 = \sqrt[5]{1 - 0.803837} = 0.721977$$
$$x_5 = \sqrt[5]{1 - 0.721977} = 0.774134$$
$$x_6 = \sqrt[5]{1 - 0.774134} = 0.742626$$
$$x_7 = \sqrt[5]{1 - 0.742626} = 0.762277$$
$$x_8 = \sqrt[5]{1 - 0.762277} = 0.750264$$
$$x_9 = \sqrt[5]{1 - 0.750264} = 0.757698$$

The convergence point is approximately $x \cong 0.75$

Fixed-Point Iteration Method: The Concept

Find the convergence point of the following sequence

$$x_{n+1} = \sqrt[5]{1 - x_n}\,, \quad x_0 = 0.5$$

Method 2: Exact solution

If the convergence point is x, we can write:

$$\lim_{n\to\infty} x_n = x$$

We can also write:

$$\lim_{n\to\infty} x_{n+1} = x$$

Now we can take $\lim_{n\to\infty}$ from both sides of the equation:

$$\lim_{n\to\infty} x_{n+1} = \lim_{n\to\infty} \sqrt[5]{1 - x_n} \quad \Rightarrow \quad x = \sqrt[5]{1 - x} \quad \Rightarrow \quad x^5 + x - 1 = 0$$

So, we conclude this important statement:

The convergence point of the sequence $x_{n+1} = \sqrt[5]{1 - x_n}$ is a root of the equation $x^5 + x - 1 = 0$.

Fixed-Point Iteration Method: The Concept

Find the convergence point of the following sequence

$$x_{n+1} = \sqrt[5]{1 - x_n}\,, \quad x_0 = 0.5$$

⇩ Approximate

We can compute the sequence

$$x_0 = 0.5$$
$$x_1 = \sqrt[5]{1 - 0.5} = 0.870551$$
$$x_2 = \sqrt[5]{1 - 0.870551} = 0.664385$$
$$x_3 = \sqrt[5]{1 - 0.664385} = 0.803837$$
$$x_4 = \sqrt[5]{1 - 0.803837} = 0.721977$$
$$x_5 = \sqrt[5]{1 - 0.721977} = 0.774134$$
$$x_6 = \sqrt[5]{1 - 0.774134} = 0.742626$$
$$x_7 = \sqrt[5]{1 - 0.742626} = 0.762277$$
$$x_8 = \sqrt[5]{1 - 0.762277} = 0.750264$$
$$x_9 = \sqrt[5]{1 - 0.750264} = 0.757698$$

The convergence point is approximately $x \cong 0.75$

⇩ Exact

To find the convergence point x we solve the following equation:

$$x^5 + x - 1 = 0$$

↓

Can we analytically solve this 5th degree equation? No.

But the root should approximately be $x \cong 0.75$

Core Idea of Fixed-Point Iteration

To find approximate root of $\boldsymbol{x^5 + x - 1 = 0}$ we can calculate sequence $\boldsymbol{x_{n+1} = \sqrt[5]{1 - x_n}}$.

This is a method for approximately solving equations!

Fixed-Point Iteration Method

Fixed-point iteration is a method of approximately solving equations.

To solve an equation in form of $f(x) = 0$, you need to follow the steps below

1) Find an interval $J = [a, b]$ that includes a root. We can use this criterion: $f(a) \cdot f(b) < 0$

2) Re-write the equation in this form: $x = g(x)$

3) Check the convergence: If the iteration remains in J, then the condition is $\max_{a \le x \le b} |g'(x)| < 1$

4) Form the sequence: $x_{n+1} = g(x_n)$ and start computation from x_0: $x_0 \to x_1 \to x_2 \to \cdots$

We will learn about the stop condition in a few slides..

Fixed-Point Iteration Method

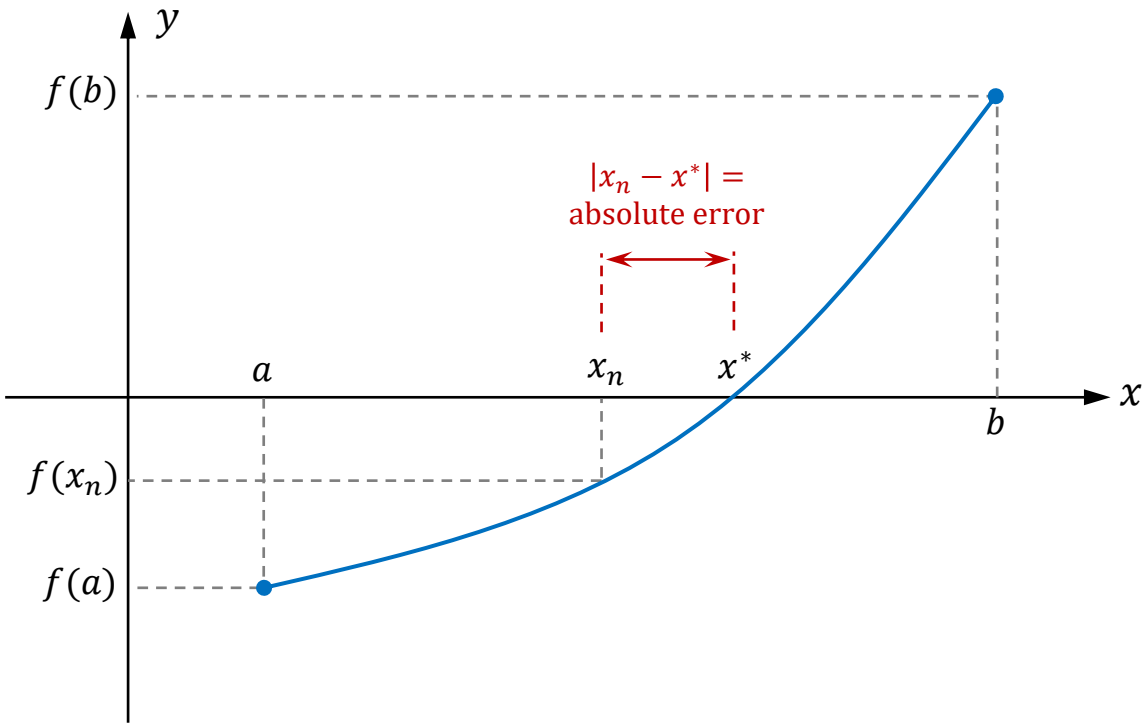
Example: Solve $x - \frac{1}{2}\cos(x) = 0$ over $x \in [0, \frac{\pi}{2}]$. Hint: take $g(x) = \frac{1}{2}\cos(x)$ and $x_0 = 0$

We follow the steps to solve the equation $f(x) = x - \frac{1}{2}\cos(x) = 0$

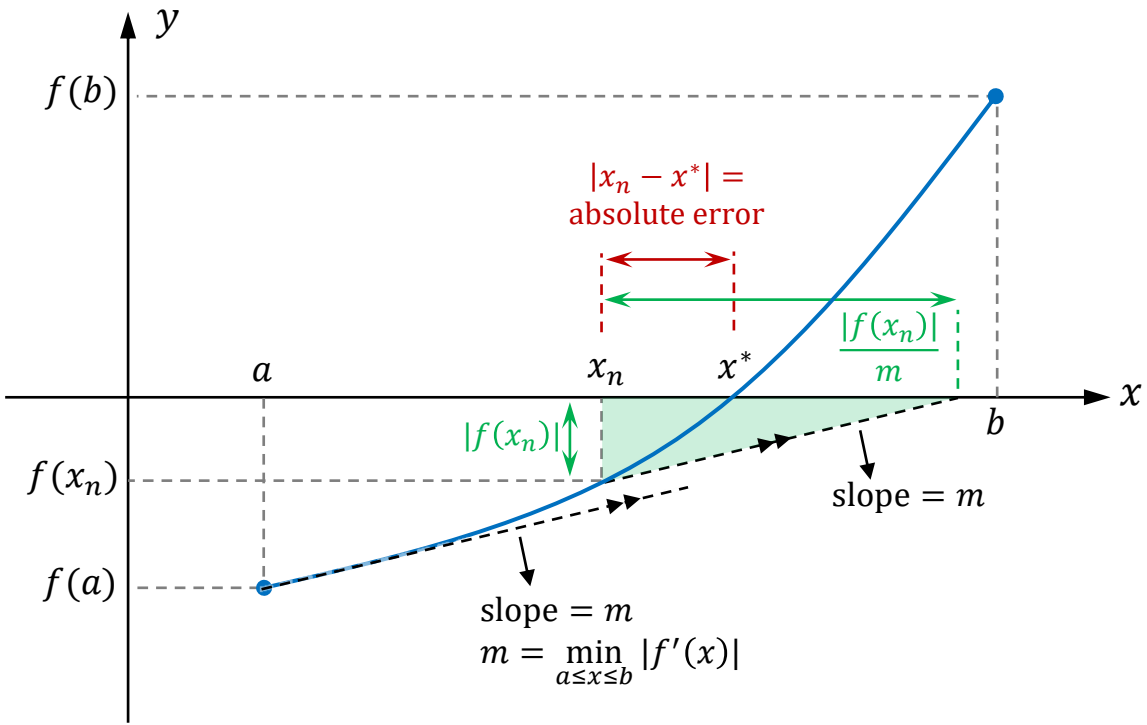
1) Find an interval $J = [a, b]$ that includes a root. We can use this criterion: $f(a) \cdot f(b) < 0$	Considering the continuous function $f(x) = x - \frac{1}{2}\cos(x)$ we obtain $f(a) = f(0) = -\frac{1}{2}$ and $f(b) = f\left(\frac{\pi}{2}\right) = \frac{\pi}{2}$, resulting in $f(a) \cdot f(b) < 0$, so there must be a root in $[0, \frac{\pi}{2}]$.
2) Re-write the equation in this form: $x = g(x)$	We can re-write the equation $x - \frac{1}{2}\cos(x) = 0$ in form of $x = \frac{1}{2}\cos(x)$.
3) Check the convergence	We have $\lvert g'(x)\rvert = \frac{1}{2}\sin(x)$ resulting in $\lvert g'(x)\rvert < 1$ for all values of x.
4) Form the sequence: $x_{n+1} = g(x_n)$ and start computation from x_0: $x_0 \to x_1 \to x_2 \to \cdots$	The sequence can be written as $x_{n+1} = \frac{1}{2}\cos(x_n)$. We start from $x_0 = 0$: $x_1 = \frac{1}{2}\cos(0) = \frac{1}{2}$, $x_2 = \frac{1}{2}\cos\left(\frac{1}{2}\right) = 0.43879128$ $x_3 = \frac{1}{2}\cos(0.43879128) = 0.45263292$, $x_4 = \frac{1}{2}\cos(0.45263292) = 0.44964938$ $x_5 = \frac{1}{2}\cos(0.44964938) = 0.45029978$, $x_6 = \frac{1}{2}\cos(0.45029978) = 0.45015833$

The root appears to be around 0.45, but where should we stop to ensure a certain level of accuracy?

Stop Condition



Stop Condition



We obtain this inequality:

$$\underbrace{|x_n - x^*|}_{\text{Absolute error}} \leq \frac{|f(x_n)|}{m}$$

Where $m = \min_{a \leq x \leq b} |f'(x)|$

We obtain the following stop condition to ensure having n correct decimals:

$$\frac{|f(x_n)|}{m} < \frac{1}{2} \times 10^{-n}$$

Fixed-Point Iteration Method with Stop Condition

NTU SINGAPORE

To solve an equation of the form $f(x) = 0$ with *K* correct decimals, you need to follow the steps below

1) Find an interval $J = [a, b]$ that includes a root. We can use this criterion: $f(a) \cdot f(b) < 0$

2) Re-write the equation in this form: $x = g(x)$

3) Check the convergence: If the iteration remains in J, then the condition is $\max_{a \le x \le b} |g'(x)| < 1$

4) Calculate $m = \min_{a \le x \le b} |f'(x)|$

5) Form the sequence: $x_{n+1} = g(x_n)$ and calculate both x_n and $\frac{|f(x_n)|}{m}$. Stop when $\frac{|f(x_n)|}{m} < \frac{1}{2} \times 10^{-K}$

Fixed-Point Iteration Method with Stop Condition

Example: Solve $x - \frac{1}{2}\cos(x) = 0$ over $x \in [0, \frac{\pi}{2}]$ with two correct decimal places. Hint: take $g(x) = \frac{1}{2}\cos(x)$ and $x_0 = 0$

1) We obtain $f(a) = f(0) = -\frac{1}{2}$ and $f(b) = f\left(\frac{\pi}{2}\right) = \frac{\pi}{2}$, resulting in $f(a) \cdot f(b) < 0$, so there must be a root in $[0, \frac{\pi}{2}]$.

2) We can re-write the equation $x - \frac{1}{2}\cos(x) = 0$ in form of $x = g(x) = \frac{1}{2}\cos(x)$.

3) We have $|g'(x)| = \frac{1}{2}\sin(x)$ resulting in $|g'^{(x)}| < 1$ for all values of x, so the convergence is guaranteed.

4) We calculate $m = \min_{0 \le x \le \pi/2} |f'(x)| = \min_{0 \le x \le \pi/2} |1 + \frac{1}{2}\sin(x)| = 1$

5) Using $x_{n+1} = \frac{1}{2}\cos(x_n)$ we calculate x_n and $\frac{|f(x_n)|}{m} = \left|x_n - \frac{1}{2}\cos(x_n)\right|$. We stop when $\frac{|f(x_n)|}{m} < \frac{1}{2} \times 10^{-2}$.

n	x_n	$\lvert f(x_n)\rvert / m$
0	0.00000000	0.50000000
1	0.50000000	0.06120872
2	0.43879128	0.01384164
3	**0.45263292**	**0.00298355**

This solution is correct to two decimal places

$x = 0.45$

$< \frac{1}{2} \times 10^{-2}$

So we can stop here!

Summary

Newton's Divided Difference Interpolation for 3 points

We want to approximate a function $f(x)$ using a second-degree polynomial $p_2(x)$.

We select 3 points from $f(x)$:

$$(x_0, f_0), \qquad (x_1, f_1), \qquad (x_2, f_2)$$

Step 1: Compute the first-divided differences

$$f[x_0, x_1] = \frac{f_1 - f_0}{x_1 - x_0}, \qquad f[x_1, x_2] = \frac{f_2 - f_1}{x_2 - x_1}$$

Step 2: Compute the second-divided differences

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$$

Step 3: Writing the interpolating polynomial

$$p_2(x) = f_0 + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1)$$

Summary

Newton's Divided Difference Interpolation for 4 points

To approximate a function $f(x)$ using a third-degree polynomial $p_3(x)$, we select 4 points from $f(x)$:

$$(x_0, f_0), \qquad (x_1, f_1), \qquad (x_2, f_2), \qquad (x_3, f_3)$$

Step 1: Compute the first-divided differences

$$f[x_0, x_1] = \frac{f_1 - f_0}{x_1 - x_0}, \qquad f[x_1, x_2] = \frac{f_2 - f_1}{x_2 - x_1}, \qquad f[x_2, x_3] = \frac{f_3 - f_2}{x_3 - x_2}$$

Step 2: Compute the second-divided differences

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}, \qquad f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}$$

Step 3: Compute the third-divided differences

$$f[x_0, x_1, x_2, x_3] = \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0}$$

Step 4: Writing the interpolating polynomial

$$p_3(x) = f_0 + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2)$$

Summary

Error of n-degree Polynomial Interpolation

Consider $f(x)$ to be $n+1$ times differentiable.

And let $p_n(x)$ be the n-th degree polynomial interpolating from the following $n+1$ points from f

$$(x_0, f_0), \quad (x_1, f_1), \quad (x_2, f_2), \quad \dots, \quad (x_n, f_n)$$

We can use the following formula to calculate an upper bound for the error of approximation

$$|f(x) - p_n(x)| \leq \frac{1}{(n+1)!} \left| \prod_{k=0}^{n} (x - x_k) \right| \max_{\xi} \left| f^{(n+1)}(\xi) \right|$$

Where $\min\{x, x_0, x_1, \dots, x_n\} \leq \xi \leq \max\{x, x_0, x_1, \dots, x_n\}$

Summary

Fixed-Point Iteration with Stop Condition

To solve an equation of the form $f(x) = 0$ with *K* correct decimals, you need to follow the steps below

1) Find an interval $J = [a, b]$ that includes a root. We can use this criteria: $f(a) \cdot f(b) < 0$

2) Re-write the equation in this form: $x = g(x)$

3) Check the convergence: If the iteration remains in J, then the condition is $\max_{a \le x \le b} |g'(x)| < 1$

4) Calculate $m = \min_{a \le x \le b} |f'(x)|$

5) Form the sequence: $x_{n+1} = g(x_n)$ and calculate both x_n and $\frac{|f(x_n)|}{m}$. Stop when $\frac{|f(x_n)|}{m} < \frac{1}{2} \times 10^{-K}$

Tutorial Questions

NTU SINGAPORE

1. Use the method of **fixed-point iteration** to solve the following equations. Give your answers correct to 3 decimal places.

   (Go through the steps given in the lecture notes.)

   (a) $x^5 = x + 0.12, \quad x \in [1, 1.5]$

   (Hint: take $g(x) = (x + 0.12)^{1/5}$)

   (b) $x^2 e^x = 1, \quad x \in [0.5, 1]$

   (Hint: take $g(x) = e^{-x/2}$)

2. Given the following table

x	0.2	0.4	0.5	0.7
$\sin x$	0.198669	??	0.479426	0.644218

   Find

   (i) the **Newton's divided difference interpolating polynomial**

   (ii) the missing value (??)

   (iii) an upper estimate for the error.

3. Given the following table

x	1.5	1.7	1.9	2.0	2.1
$x^{1/3}$	1.144714	1.193483	1.238562	??	1.280579

   Find

   (i) the **Newton's divided difference interpolating polynomial**

   (ii) the missing value (??)

   (iii) an upper estimate for the error.